DressSales Forecast Prediction and Animation

- **Project Details**
  - **By:** Komal Shahid
  - **Due Date:** Aug 10, 2024
  - **DSC 630**
  -  Predictive Analytics

Project Details

Import Libraries

In [1]:
```
import numpy as np
import pandas as pd
import seaborn as sns
from pprint import pprint
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder
from sklearn.metrics import mean_squared_error, mean_absolute_error
import plotly.express as px
```

In [2]:
```
import warnings

# Suppress specific warnings
warnings.filterwarnings("ignore", category=UserWarning)
```

Dataset Details

- **Attributes Dataset:**
  - The dataset used for this project is sourced from Kaggle. It contains information about various attributes of dresses, such as style, price, rating, size, season, neckline, sleeve length, material, fabric type, decoration, pattern type, and recommendation.
  - Link to Dataset

In [3]:
```
# Load the datasets
attributes = pd.read_csv("data/Attribute dataset.csv")
sales = pd.read_csv("data/DressSales.csv")
```

Data Exploration

- **Attributes DataFrame:**
  - The `attributes` DataFrame contains information about the attributes of dresses.
  - It has the following columns:
    - Dress_ID: Unique identifier for each dress

- Style: Style of the dress
- Price: Price category of the dress
- Rating: Rating of the dress
- Size: Size category of the dress
- Season: Season in which the dress is suitable
- NeckLine: Neckline style of the dress
- SleeveLength: Sleeve length of the dress
- Material: Material used in the dress
- FabricType: Fabric type of the dress
- Decoration: Decoration style of the dress
- Pattern Type: Pattern type of the dress
- Recommendation: Recommendation for the dress (0 or 1)

- **Summary of Attributes DataFrame:**
  - The `attributes` DataFrame has a total of 479 entries and 13 columns.
  - It contains a combination of numeric and categorical columns.
  - The memory usage of the DataFrame is 48.8 KB.

In [4]:
```
# Display the first few rows of the datasets
print(attributes.head())
print(sales.head())
```

```
     Dress_ID    Style    Price  Rating Size  Season NeckLine SleeveLength  \
0  1006032852     Sexy      Low     4.6    M  Summer   o-neck    sleevless
1  1212192089   Casual      Low     0.0    L  Summer   o-neck        Petal
2  1190380701  vintage     High     0.0    L  Automn   o-neck         full
3   966005983    Brief  Average     4.6    L  Spring   o-neck         full
4   876339541     cute      Low     4.5    M  Summer   o-neck    butterfly

        Material FabricType  Decoration Pattern Type  Recommendation
0            NaN    chiffon     ruffles       animal               1
1     microfiber        NaN     ruffles       animal               0
2       polyster        NaN         NaN        print               0
3           silk    chiffon  embroidary        print               1
4  chiffonfabric    chiffon         bow          dot               0
     Dress_ID  29-08-2013  31-08-2013  09-02-2013  09-04-2013  09-06-2013  \
0  1006032852        2114        2274        2491        2660        2727
1  1212192089         151         275         570         750         813
2  1190380701           6           7           7           7           8
3   966005983        1005        1128        1326        1455        1507
4   876339541         996        1175        1304        1396        1432

   09-08-2013  09-10-2013 09-12-2013 14-09-2013  ... 24-09-2013 26-09-2013  \
0        2887        2930       3119       3204  ...       3554     3624.0
1        1066        1164       1558       1756  ...       2710     2942.0
2           8           9         10         10  ...         11       11.0
3        1621        1637       1723       1746  ...       1878     1892.0
4        1559        1570       1638       1655  ...       2032     2156.0

  28-09-2013 30-09-2013  10-02-2013  10-04-2013  10-06-2013  10-08-2013  \
0       3706     3746.0      3795.0      3832.0        3897      3923.0
1       3258     3354.0      3475.0      3654.0        3911      4024.0
2         11       11.0        11.0        11.0          11        11.0
3       1914     1924.0      1929.0      1941.0        1952      1955.0
4       2252     2312.0      2387.0      2459.0        2544      2614.0

   10-10-2013  10-12-2013
0      3985.0        4048
```

```
1      4125.0        4277
2        11.0          11
3      1959.0        1963
4      2693.0        2736

[5 rows x 24 columns]
```

In [5]:
```
# # Display information about the datasets
# print(attributes.info())
# print(sales.info())
```

In [6]:
```
categorical_columns = [
    "Price",
    "Material",
    "Decoration",
    "Pattern Type",
    "SleeveLength",
    "Season",
    "Size",
    "Style",
    "NeckLine",
    "FabricType",
]


# a dictionary to store the column names and unique values
unique_values_dict = {}


for column in categorical_columns:
    unique_vals = (
        attributes[column].unique().tolist()
    )  # Get unique values for each column in *attributes* df

    # Add the column name and unique values to the dictionary
    unique_values_dict[column] = unique_vals

# Convert the dictionary to a DataFrame for better readability
unique_values_df = pd.DataFrame(
    list(unique_values_dict.items()), columns=["Column_Name", "Unique_Values"]
)

# Display the DataFrame
pd.set_option(
    "display.max_colwidth", None
)  # to display all of the values in the columns
unique_values_df
```

Out[6]:

	Column_Name	Unique_Values
0	Price	[Low, High, Average, Medium, very-high, nan]
1	Material	[nan, microfiber, polyster, silk, chiffonfabric, cotton, nylon, other, milksilk, linen, rayon, lycra, mix, acrylic, spandex, lace, modal, cashmere, viscos, sill, wool, model, shiffon]
2	Decoration	[ruffles, nan, embroidary, bow, beading, lace, sashes, hollowout, pockets, sequined, applique, button, Tiered, rivet, feathers, flowers, pearls, pleat, crystal, ruched, draped, tassel, plain, cascading, none]
3	Pattern Type	[animal, print, dot, solid, nan, patchwork, striped, geometric, plaid, leopard, floral, character, splice, leapord, none]
4	SleeveLength	[sleevless, Petal, full, butterfly, short, threequarter, halfsleeve, cap-sleeves, turndowncollor, threequater, capsleeves, sleeveless, sleeevless, half, urndowncollor, thressqatar, nan, sleveless]
5	Season	[Summer, Automn, Spring, Winter, spring, winter, nan, Autumn]
6	Size	[M, L, XL, free, S, small, s]
7	Style	[Sexy, Casual, vintage, Brief, cute, bohemian, Flare, party, sexy, Novelty, work, OL, fashion]

8	NeckLine	[o-neck, v-neck, boat-neck, peterpan-collor, ruffled, turndowncollor, slash-neck, mandarin-collor, open, sqare-collor, Sweetheart, sweetheart, nan, Scoop, halter, backless, bowneck]
9	FabricType	[chiffon, nan, broadcloth, jersey, other, batik, satin, flannael, worsted, woolen, poplin, dobby, knitting, flannel, tulle, sattin, organza, lace, Corduroy, wollen, knitted, shiffon, terry]

Data Processing

- **Missing Values Handling:**
  - The `attributes` DataFrame has missing values in some columns.
  - The missing values will be handled using appropriate techniques, such as imputation or dropping columns.
- **Data Cleaning:**
  - The data in some columns will be cleaned and corrected to ensure consistency and accuracy.

In [7]:
```
# replace the spelling mistakes in 'SleeveLength
corrections = {
    "sleevless": "sleeveless",
    "sleeevless": "sleeveless",
    "sleveless": "sleeveless",
    "urndowncollor": "turndowncollar",
    "thressqatar": "threequarter",
    "threequater": "threequarter",
    "halfsleeve": "halfsleeve",
    "turndowncollor": "turndowncollar",  # American English,
    "cap-sleeves": "capsleeves",
    "Automn": "Autumn",
    "spring": "Spring",
    "winter": "Winter",
    "shiffon": "chiffon",
    "chiffonfabric": "chiffon",
    "leapord": "leopard",
    "wollen": "woolen",
    "flannael": "flannel",
    "sattin": "satin",
    "M": "Medium",
    "L": "Large",
    "XL": "Extra Large",
    "free": "Free",
    "S": "Small",
    "s": "Small",
    "small": "Small",
    "Medium": "Average",
}

# List of columns you want to correct
columns = [
    "Material",
    "Pattern Type",
    "Style",
    "FabricType",
    "Season",
    "SleeveLength",
    "Size",
    "Price",
]

# Use the replace function to correct the spellings and convert all the strings to lower
```

```
for col in columns:
    attributes[col] = attributes[col].replace(corrections).str.lower()

# attributes.head()
```

In [8]:
```
# Calculate and print the percentage of each category in 'Size' column
size_percentages = attributes["Size"].value_counts(normalize=True) * 100
print(size_percentages)
```

```
Size
medium         35.699374
free           34.446764
large          19.415449
small           7.515658
extra large     2.922756
Name: proportion, dtype: float64
```

In [9]:
```
# Convert the Sales dataset values into Float

columns_to_exclude = ["Dress_ID"]
# Get the rest of the columns
columns_to_convert = [col for col in sales.columns if col not in columns_to_exclude]

# Iterate over the rest of the columns in your DataFrame
for col in columns_to_convert:
    # Use to_numeric to convert non-numeric values to NaN
    sales[col] = pd.to_numeric(sales[col], errors="coerce")
    # Convert column to float type
    sales[col] = sales[col].astype(float)
```

In [10]:
```
# total sales of each dress for the season
def season_sum(data, dates):
    return data[dates].sum(axis=1)


# Define the dates for each season
summer_dates = ["29-08-2013", "31-08-2013", "09-06-2013", "09-08-2013", "10-06-2013"]
autumn_dates = [
    "09-10-2013",
    "14-09-2013",
    "16-09-2013",
    "18-09-2013",
    "20-09-2013",
    "22-09-2013",
    "24-09-2013",
    "28-09-2013",
]
winter_dates = ["09-02-2013", "10-12-2013"]
spring_dates = ["09-04-2013"]

# Add the season sums to the DataFrame for Seasonal Sales Analysis
sales["Summer"] = season_sum(sales, summer_dates)
sales["Autumn"] = season_sum(sales, autumn_dates)
sales["Winter"] = season_sum(sales, winter_dates)
sales["Spring"] = season_sum(sales, spring_dates)
```

In [11]:
```
# merge the data frames
dress_sales_df = pd.merge(attributes, sales, on="Dress_ID")
dress_sales_df.head(2)
```

Out[11]:

	Dress_ID	Style	Price	Rating	Size	Season	NeckLine	SleeveLength	Material	FabricType	...	10-2(
0	1006032852	sexy	low	4.6	medium	summer	o-neck	sleeveless	NaN	chiffon	...	379

1	1212192089	casual	low	0.0	large	summer	o-neck		petal	microfiber		NaN	...	347

2 rows × 40 columns

Missing Values Handling

In [12]:
```
null_counts = dress_sales_df.isna().sum()
null_pairs = [
    (col, int(count)) for col, count in zip(null_counts.index, null_counts.values)
]
pprint(null_pairs, compact=True, width=150)
```

```
[('Dress_ID', 0), ('Style', 0), ('Price', 2), ('Rating', 0), ('Size', 0), ('Season', 2),
('NeckLine', 3), ('SleeveLength', 2), ('Material', 119),
 ('FabricType', 256), ('Decoration', 224), ('Pattern Type', 102), ('Recommendation', 0),
('29-08-2013', 0), ('31-08-2013', 0), ('09-02-2013', 0),
 ('09-04-2013', 0), ('09-06-2013', 0), ('09-08-2013', 0), ('09-10-2013', 0), ('09-12-201
3', 1), ('14-09-2013', 1), ('16-09-2013', 1),
 ('18-09-2013', 1), ('20-09-2013', 1), ('22-09-2013', 1), ('24-09-2013', 0), ('26-09-201
3', 222), ('28-09-2013', 0), ('30-09-2013', 257),
 ('10-02-2013', 259), ('10-04-2013', 258), ('10-06-2013', 0), ('10-08-2013', 255), ('10-
10-2013', 255), ('10-12-2013', 0), ('Summer', 0),
 ('Autumn', 0), ('Winter', 0), ('Spring', 0)]
```

Sales Column Missing Values Processing

In [13]:
```
# Drop the column with most missing values but not including Sales
sales_columns = [
    "29-08-2013",
    "31-08-2013",
    "09-02-2013",
    "09-04-2013",
    "09-06-2013",
    "09-08-2013",
    "09-10-2013",
    "09-12-2013",
    "14-09-2013",
    "16-09-2013",
    "18-09-2013",
    "20-09-2013",
    "22-09-2013",
    "24-09-2013",
    "26-09-2013",
    "28-09-2013",
    "30-09-2013",
    "10-02-2013",
    "10-04-2013",
    "10-06-2013",
    "10-08-2013",
    "10-10-2013",
    "10-12-2013",
]
# Fill NA/NaN values in 'Sales' columns with 0's instead of dropping the columns
dress_sales_df[sales_columns] = dress_sales_df[sales_columns].fillna(0)
```

In [14]:
```
#  Impute missing values with the mean (or median for categorical data) for dataframe be
def fill_missing_values(df: pd.DataFrame):
    """Imputes missing values in a DataFrame with mean (numerical) or mode (categorical)

    Args:
        df: The DataFrame to impute missing values in.
```

```
    Returns:
        A new DataFrame with missing values imputed.
    """
    new_df = df.copy()  # Create a copy to avoid modifying original DataFrame

    for col in new_df.columns:
        if pd.api.types.is_numeric_dtype(new_df[col]):
            # numerical columns with mean
            new_df[col].fillna(new_df[col].mean())
        else:
            # categorical columns with mode
            new_df[col].fillna(new_df[col].mode()[0])
    return new_df
```

In [15]:
```
def create_lags(df, lag) -> pd.DataFrame:
    """Creates lagged features for a given DataFrame.
    lagged sales data can significantly improve the performance of an ARIMA model

    Args:
      df: The DataFrame containing the data.
      lags: The number of lags to create.

    Returns:
      The DataFrame with added lagged features.
    """
    df_copy = df.copy()
    lagged_data = pd.DataFrame(index=df_copy.index)
    for col in df_copy.columns:
        if col.startswith("Sales_"):
            for i in range(1, lag + 1):
                lagged_col_name = f"{col}_lag_{i}"
                if lagged_col_name in lagged_data.columns:
                    raise ValueError(f"Column {lagged_col_name} already exists in the Da
                lagged_data[lagged_col_name] = df_copy[col].shift(i)
                assert lagged_data[lagged_col_name].ndim == 1, f"Column {lagged_col_name
    return lagged_data
```

In [16]:
```
def create_cyclic_features_month(df):
    """Creates cyclic features for the month.
    cyclic features  can help understand seasonal patterns.


    Args:
      data: The DataFrame containing the data.

    Returns:
      The DataFrame with added cyclic features.
    """
    df_copy = df.copy()
    cyclic_data = pd.DataFrame(index=df_copy.index)
    cyclic_data['month'] = df_copy.index.month
    cyclic_data['month_sin'] = np.sin(2 * np.pi * cyclic_data['month'] / 12)
    cyclic_data['month_cos'] = np.cos(2 * np.pi * cyclic_data['month'] / 12)
    assert cyclic_data['month_sin'].ndim == 1, "Column month_sin is not one-dimensional"
    assert cyclic_data['month_cos'].ndim == 1, "Column month_cos is not one-dimensional"
    return cyclic_data
```

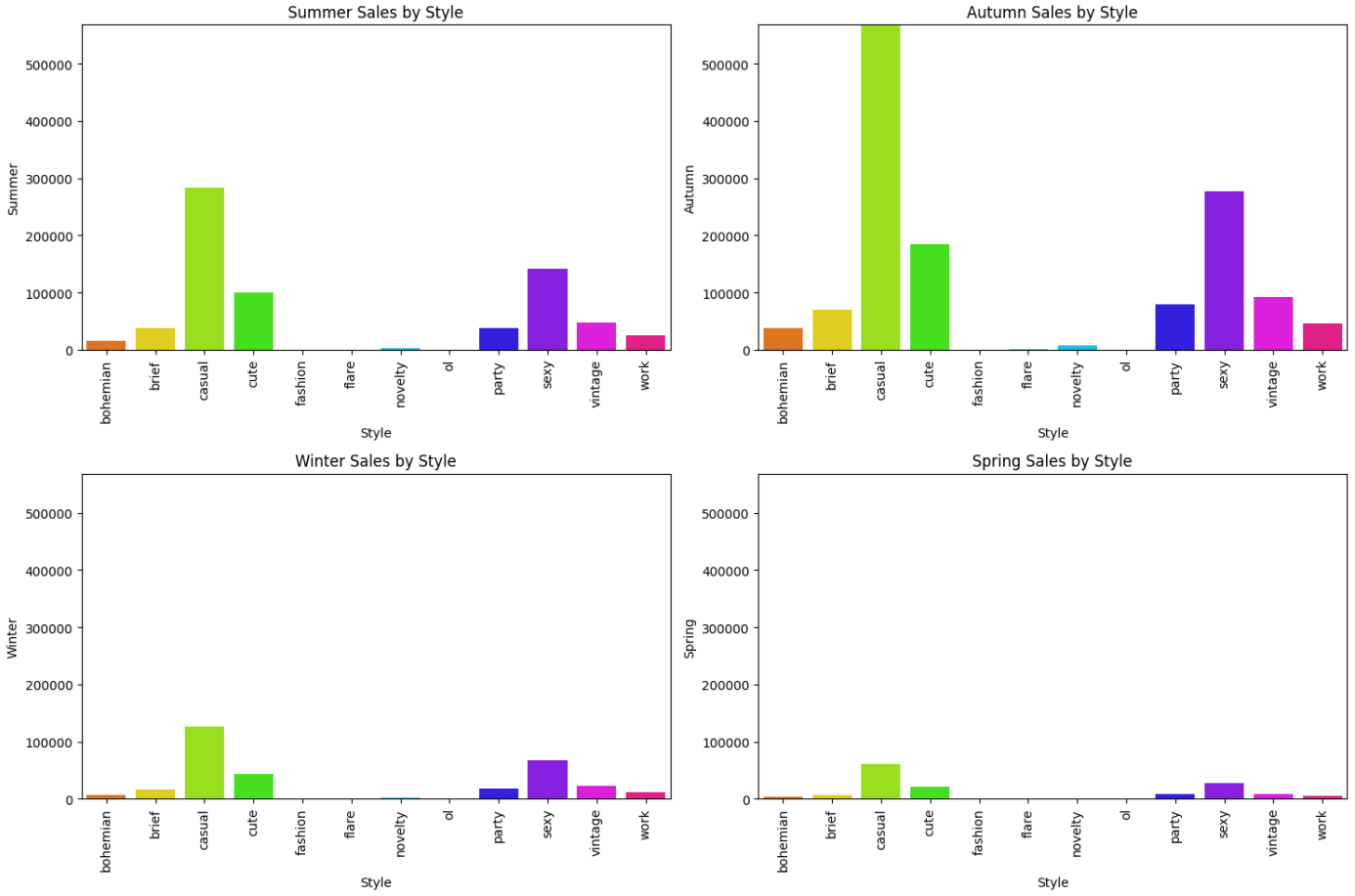
Data Visualization

- **Seasonal Sales Analysis:**

    - The sales data will be analyzed based on different seasons.
    - The sales for each season will be visualized using bar plots.
- **Style Distribution:**
    - The distribution of dress styles will be visualized using a pie chart.
- **Recommendation Analysis:**
    - It analyzes the relationship between dress recommendations and season using a color-coded scatter plot.

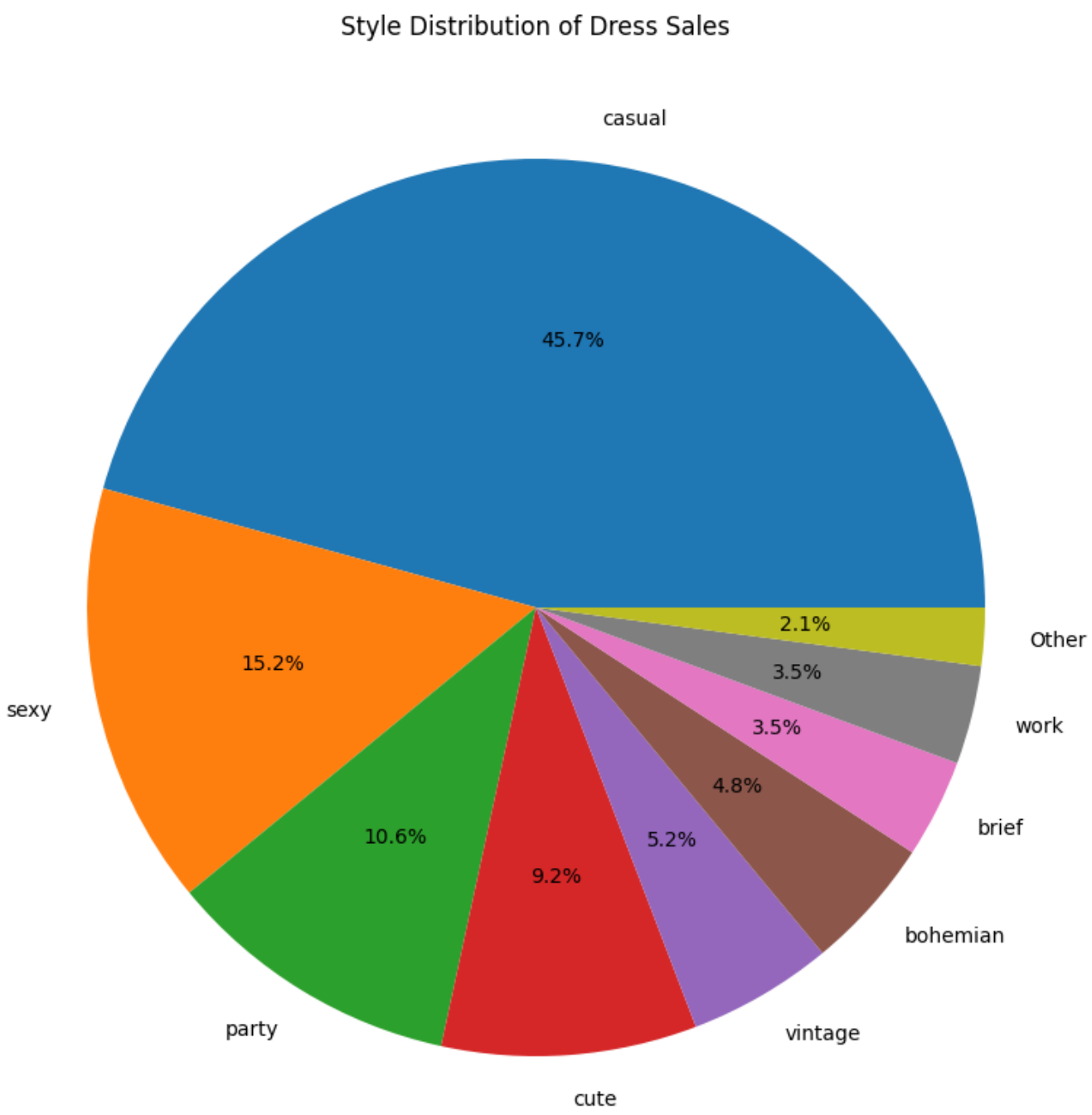
In [17]:
```
# Group by 'Style' and calculate the sum of sales for each season
grouped = dress_sales_df.groupby("Style")[
    ["Summer", "Autumn", "Winter", "Spring"]
].sum()
max_y = grouped[["Summer", "Autumn", "Winter", "Spring"]].values.max()

# Create subplots for each season
fig, axs = plt.subplots(2, 2, figsize=(15, 10))

palette = sns.color_palette("hsv", len(grouped.index))  # color palette for the graphs

# Summer sales
sns.barplot(
    x=grouped.index,
    y=grouped["Summer"],
    hue=grouped.index,
    ax=axs[0, 0],
    palette=palette,
)
axs[0, 0].set_title("Summer Sales by Style")
axs[0, 0].tick_params(axis="x", rotation=90)
axs[0, 0].set_ylim([0, max_y])

# Autumn sales
sns.barplot(
    x=grouped.index,
    y=grouped["Autumn"],
    hue=grouped.index,
    ax=axs[0, 1],
    palette=palette,
)
axs[0, 1].set_title("Autumn Sales by Style")
axs[0, 1].tick_params(axis="x", rotation=90)
axs[0, 1].set_ylim([0, max_y])

# Winter sales
sns.barplot(
    x=grouped.index,
    y=grouped["Winter"],
    hue=grouped.index,
    ax=axs[1, 0],
    palette=palette,
)
axs[1, 0].set_title("Winter Sales by Style")
axs[1, 0].tick_params(axis="x", rotation=90)
axs[1, 0].set_ylim([0, max_y])

# Spring sales
sns.barplot(
    x=grouped.index,
    y=grouped["Spring"],
    hue=grouped.index,
    ax=axs[1, 1],
```

```
    palette=palette,
)
axs[1, 1].set_title("Spring Sales by Style")
axs[1, 1].tick_params(axis="x", rotation=90)
axs[1, 1].set_ylim([0, max_y])

# Adjust the layout
plt.tight_layout()
plt.show()
```

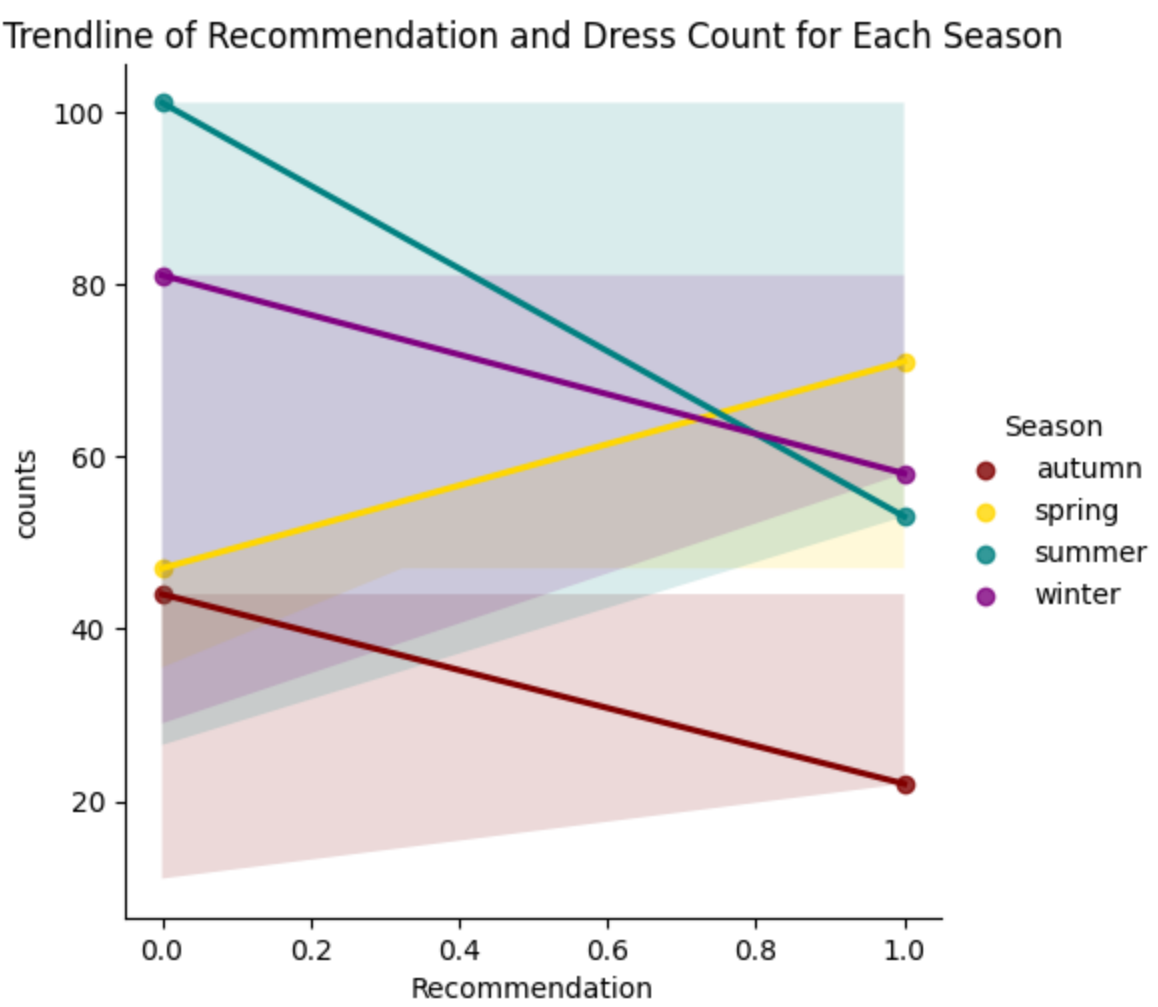


In [18]:

```
# Dress Style Distribution
style_counts = dress_sales_df["Style"].value_counts()

# Filter out styles with less than a certain threshold to make the pie chart more readab
threshold = 10
prominent_styles = style_counts[style_counts >= threshold]

# call all the Styles with less than 3.5 threshold as "Other"
if style_counts[style_counts < threshold].sum() > 0:
    other = pd.Series([style_counts[style_counts < threshold].sum()], index=["Other"])
    prominent_styles = pd.concat([prominent_styles, other])

# Create the pie chart
plt.figure(figsize=(14, 10))
plt.pie(prominent_styles, labels=prominent_styles.index, autopct="%1.1f%%")
plt.title("Style Distribution of Dress Sales")
plt.show()
```



In [19]:
```
#  a color map for the 'Season'
colors = {"spring": "gold", "summer": "teal", "autumn": "maroon", "winter": "purple"}
count = (
    dress_sales_df.groupby(["Recommendation", "Season"])
    .size()
    .reset_index(name="counts")
)

# Create lmplot
sns.lmplot(
    x="Recommendation",
    y="counts",
    data=count,
    hue="Season",
    palette=colors,
    fit_reg=True,
)
plt.title("Trendline of Recommendation and Dress Count for Each Season")
plt.show()
```



In [20]:

```
# Aggregate sales data across all dates and seasons
sales_columns = dress_sales_df.columns[
    13:32
]  # Assuming sales columns start from the 11th column
dress_sales_df[sales_columns] = dress_sales_df[sales_columns].apply(
    pd.to_numeric, errors="coerce"
)
dress_sales_df["Total_Sales"] = dress_sales_df[sales_columns].sum(axis=1)

# Group by Dress_ID to get total recommendations
dress_sales_df["Total_Recommendations"] = dress_sales_df.groupby("Dress_ID")[
    "Recommendation"
].transform("sum")

# Filter and sort data to get top 10 dresses with highest total recommendations
top_dresses = (
    dress_sales_df.groupby("Dress_ID")["Total_Recommendations"].sum().nlargest(10).index
)
top_dress_attributes = dress_sales_df[dress_sales_df["Dress_ID"].isin(top_dresses)][
    [
        "Dress_ID",
        "Total_Sales",
        "Total_Recommendations",
        "Style",
        "NeckLine",
        "Season",
        "Price",
    ]
].drop_duplicates(subset=["Dress_ID"])


# Create scatter plot with bubble sizes representing total sales
```

```
fig = px.scatter(
    top_dress_attributes,
    x="Total_Sales",
    y="Total_Recommendations",
    size="Total_Sales",
    title="Top 10 Dresses with Highest Total Recommendations",
    labels={
        "Total_Sales": "Total Sales",
        "Total_Recommendations": "Total Recommendations",
    },
    hover_name="Dress_ID",
    color=px.colors.qualitative.Plotly,  # Use a color palette
)

# Update layout to move legend and increase width
fig.update_layout(
    plot_bgcolor="teal",
    paper_bgcolor="white",
    width=1200,  # Increase the width of the graph area
    showlegend=False,
)

# Add annotations for each point without arrows
for i, row in top_dress_attributes.iterrows():
    annotation_text = (
        f"ID: {row['Dress_ID']}<br>"
        f"Style: {row['Style']}<br>"
        f"Neckline: {row['NeckLine']}<br>"
        f"Season: {row['Season']}<br>"
        f"Price: {row['Price']}"
    )

    if i % 2 == 0:
        ax, ay = 20, 80  # Position above
    else:
        ax, ay = -30, -80  # Position below
    fig.add_annotation(
        x=row["Total_Sales"],
        y=row["Total_Recommendations"],
        text=annotation_text,
        showarrow=True,
        arrowhead=1,
        bgcolor="lightyellow",
        borderpad=1.7,
        font=dict(
            size=7, color="black", family="TimesNewRoman"  # Change the font family
        ),
        ax=ax,  # Shift annotation horizontally
        ay=ay,  # Shift annotation vertically
    )


fig.show()
```

In [21]:

```
# Separate the seasonal and date columns
seasons = ["Summer", "Autumn", "Winter", "Spring"]
date_cols = [col for col in sales.columns if col not in seasons]


# Melt the sales data separately
sales_melted = pd.melt(
    sales[date_cols], id_vars=["Dress_ID"], var_name="Date", value_name="Sales"
)
print(
```

```
    f"salesdf melted for dates have: {sales_melted.columns} columns and the shape is {sa
)

# Convert the 'Date' column to datetime format
sales_melted["Date"] = pd.to_datetime(
    sales_melted["Date"], format="%d-%m-%Y", errors="coerce"
)

print(
    f"sales_melted df: \n {sales_melted.head()}. \nThe null values are:\n{sales_melted.i
)
```

```
salesdf melted for dates have: Index(['Dress_ID', 'Date', 'Sales'], dtype='object') colu
mns and the shape is (11017, 3) .
sales_melted df:
      Dress_ID       Date   Sales
0  1006032852 2013-08-29  2114.0
1  1212192089 2013-08-29   151.0
2  1190380701 2013-08-29     6.0
3   966005983 2013-08-29  1005.0
4   876339541 2013-08-29   996.0.
The null values are:
Dress_ID       0
Date           0
Sales       1512
dtype: int64
```

In [22]:

```
# Merge with attributes df and fill in the missing values for Sales only with 0.
sales_data = pd.merge(sales_melted, attributes, on="Dress_ID", how="left")
sales_data.dropna(subset=["Price"], inplace=True)
sales_data.fillna({"Sales": 0}, inplace=True)
print(f"Newly merge df with combined dataframes have {sales_data.shape}.")
```

```
Newly merge df with combined dataframes have (10971, 15).
```

In [23]:

```
# Create new features
sales_data["On_Sale"] = sales_data["Price"].apply(lambda x: 1 if x == "low" else 0)
# Handle missing values in sales_cleaned (impute with mean)
sales_data = fill_missing_values(sales_data)
print(
    f"sales_data has the following:\n{sales_data.dtypes}",
    f"\n\nAny Null values in model ready data result : {sales_data.isna().sum().all()}."
)
```

```
sales_data has the following:
Dress_ID                   int64
Date               datetime64[ns]
Sales                     float64
Style                      object
Price                      object
Rating                    float64
Size                       object
Season                     object
NeckLine                   object
SleeveLength               object
Material                   object
FabricType                 object
Decoration                 object
Pattern Type               object
Recommendation              int64
On_Sale                     int64
dtype: object

Any Null values in model ready data result : False.
```

Building the ARIMA Model:

- Before building the **ARIMA** model, we need to ensure that our data is stationary. We can use the Augmented Dickey-Fuller test for this. If the data is not stationary, we can use differencing techniques to make it stationary.
- we need to make sure data is stationary.

Data Preparation for ARIMA (Stationary Dataset)

-- Since Data is going to be our *Index*, we need to make sure it is present and the values are unique.

Data Preparation Process

1. **Load the Data**: Import the dataset and convert the 'Date' column to datetime format.
2. **Set the Index**: Ensure the 'Date' column is set as the index.
3. **Create Lagged Features**: Generate lagged features for the 'Sales__' columns.
4. **Drop Duplicates**: Remove any duplicate rows based on the 'Date' column.
5. **Split the Data**: Divide the data into training and testing sets.
6. **Set Frequency**: Ensure the frequency of the `DateTimeIndex` is set.

In [24]:
```
if "Date" not in sales_data.columns:
    raise KeyError("'Date' column is missing from the DataFrame")
```

***sales_data** is in LONG format as it has duplicate dates for unique dress_ID. ARIMA data processing steps would drop the duplicates for index so all the data would be last. To mitigate the lost of the rows, we can pivot the dress_ID data into each column.*

In [25]:
```
pivoted_sales = sales_data.pivot_table(
    index="Date", columns="Dress_ID", values="Sales", aggfunc="sum"
)
pivoted_sales.columns = [f"Sales_{col}" for col in pivoted_sales.columns]
pivoted_sales.shape
```

Out[25]:
```
(23, 477)
```

In [26]:
```
other_cols = (
    sales_data.drop(["Sales"], axis=1)
    .drop_duplicates(subset=["Date"])
    .reset_index(drop=True)
)
merged_data = pd.merge(pivoted_sales.reset_index(), other_cols, on="Date", how="left")
# merged_data["Date"] = pd.to_datetime(merged_data["Date"])
# merged_data.set_index("Date", inplace=True)
```

In [27]:
```
categorical_cols = [
    "Style",
    "Size",
    "Season",
    "NeckLine",
    "SleeveLength",
    "Material",
    "FabricType",
    "Decoration",
    "Pattern Type",
]

encoder = OneHotEncoder(sparse_output=False)
encoded_data = encoder.fit_transform(merged_data[categorical_cols])
```

```
encoded_df = pd.DataFrame(
    encoded_data, columns=encoder.get_feature_names_out(categorical_cols)
)
```

In [28]:
```
ordinal_encoder = OrdinalEncoder(categories=[["low", "average", "high", "very-high"]])
merged_data["Price_encoded"] = ordinal_encoder.fit_transform(merged_data[["Price"]])
merged_data.drop("Price", axis=1, inplace=True)
```

In [29]:
```
final_data = pd.concat([merged_data.drop(categorical_cols, axis=1), encoded_df], axis=1)
```

In [30]:
```
# Ensure 'Date' column exists and remove any duplicate 'Date' columns
if "Date" in final_data.columns:
    train = final_data.loc[:, ~final_data.columns.duplicated()]

# Convert 'Date' to datetime format, then drop duplicates and set the index
final_data["Date"] = pd.to_datetime(final_data["Date"], errors="coerce")
final_data = final_data.drop_duplicates(subset="Date")
print(f"final_data date column type is {final_data.dtypes['Date']}.")
final_data.set_index("Date", inplace=True)
final_data.index = pd.to_datetime(final_data.index)

# Split the data into training and testing sets
split_point = int(len(final_data) * 0.8)
train, test = final_data[:split_point], final_data[split_point:]

# Convert the index of train and test to datetime
train.index = pd.to_datetime(train.index)
test.index = pd.to_datetime(test.index)
print(
    f"train index dtype is: {train.index.dtype}, test index dtype is: {test.index.dtype}
)
```

```
final_data date column type is datetime64[ns].
train index dtype is: datetime64[ns], test index dtype is: datetime64[ns].
```

FEATURE SELECTION

SALES ATTRIBUTES:

We will select relevant features for the model, including lagged sales data and cyclic features for the month.

In [31]:
```
# Infer and set the frequency of the DateTimeIndex
train.index.freq = pd.infer_freq(train.index)
#dimensions of the train and test data
print(f"train data shape: {train.shape}, test data shape: {test.shape}.")
```

```
train data shape: (18, 491), test data shape: (5, 491).
```

Model Building and Evaluation

Grid Search for ARIMA Order

In [32]:
```
# Grid Search for ARIMA order
def grid_search_arima(series, p_values, d_values, q_values):
    best_aic = np.inf
    best_order = None
    best_model = None
```

```
    for p in p_values:
        for d in d_values:
            for q in q_values:
                try:
                    model = ARIMA(series, order=(p, d, q))
                    results = model.fit()
                    if results.aic < best_aic:
                        best_aic = results.aic
                        best_order = (p, d, q)
                        best_model = results
                except:
                    continue

    return best_order, best_model
```

Step 3: Fit ARIMA

In [ ]:

```
# Define p, d, q ranges
p_values = range(0, 4)
d_values = range(0, 2)
q_values = range(0, 4)

# Perform grid search for each 'Sales_' column
best_orders = {}

# Create lagged data and cyclic features for all 'Sales_' columns
sales_columns = [col for col in train.columns if col.startswith("Sales_")]
lagged_data = create_lags(train[sales_columns].copy(), 3)
cyclic_data = create_cyclic_features_month(train.copy())

# Collect all new columns into a list
new_columns = [lagged_data, cyclic_data]

# Concatenate lagged data and cyclic features at once
train = pd.concat([train] + new_columns, axis=1)

# Create a copy to defragment the DataFrame
train = train.copy()

# Perform grid search for each 'Sales_' column
final_evaluation_metrics = []
for col in sales_columns:
    series = train[col].dropna()

    # Ensure the series has no NaN or infinite values
    if not np.isfinite(series).all():
        series = series.replace([np.inf, -np.inf], np.nan).dropna()

    # Handle zero or negative values by adding a small constant
    series = series + 1e-10

    best_order, best_model = grid_search_arima(
        series, p_values, d_values, q_values
    )
    best_orders[col] = best_order
    metrics = {
        "Column": col,
        "Best Order": best_order,
        "AIC": best_model.aic,
    }
    final_evaluation_metrics.append(metrics)

# Save final evaluation metrics to CSV
```

```
final_evaluation_df = pd.DataFrame(final_evaluation_metrics)
final_evaluation_df.to_csv('final_evaluation_metrics.csv', index=False)
```

In [ ]:
```
# Convert evaluation metrics to DataFrame
evaluation_df = pd.DataFrame(evaluation_metrics)
# Save DataFrame to CSV
evaluation_df.to_csv("results/arima_evaluation_metrics.csv", index=False)

# print the DataFrame
print(evaluation_df.head())
```

Step 4: EVALUATION METRICS:

Column: The name of the column being evaluated.

Best Order: The best (p, d, q) order for the ARIMA model.

AIC: The Akaike Information Criterion value for the best ARIMA model.

In [ ]:
```
# Read the CSV file into a DataFrame
evaluation_df = pd.read_csv("results/arima_evaluation_metrics.csv")

# Create a scatter plot for AIC values
fig = px.scatter(
    evaluation_df,
    x="Column",
    y="AIC",
    color="Dickey-Fuller Test",
    title="ARIMA Model Evaluation Metrics",
    labels={"AIC": "AIC Value", "Column": "Sales Column"},
    hover_data=["Best Order", "ADF Statistic", "p-value"],
)

# Show the plot
fig.show()
```

In [ ]:
```
# Convert evaluation metrics to DataFrame for better readability
evaluation_df = pd.DataFrame(evaluation_metrics)
print(evaluation_df)
```

Plotting the forecasting results:

Interactive Plot:

```
- The `create_interactive_plot` function creates an interactive plot
with a dropdown menu.
- Each dropdown option corresponds to a different sales column.
- The plot title updates to include dress attributes (e.g., material
type, sleeve length).
```

Dropdown Menu:

```
- The dropdown menu allows users to select different sales columns.
- The plot updates to show the actual and forecasted sales for the
selected column.
```

In [ ]:
```
# Visualization with Plotly
```

```python
def create_interactive_plot():
    fig = make_subplots()

    # Add traces for each sales column
    for col in forecasts.keys():
        actual_data = test[col]
        forecast_data = forecasts[col]

        # Extract dress attributes
        dress_id = col.split("_")[1]
        dress_attributes = final_data.loc[final_data["Dress_ID"] == int(dress_id)].iloc[
            0
        ]
        attributes_text = f"Material: {dress_attributes['Material']}, Sleeve Length: {dr

        fig.add_trace(
            go.Scatter(
                x=actual_data.index,
                y=actual_data,
                mode="lines",
                name=f"Actual Sales {col}",
                visible=False,
            )
        )
        fig.add_trace(
            go.Scatter(
                x=forecast_data.index,
                y=forecast_data,
                mode="lines",
                name=f"Forecasted Sales {col}",
                line=dict(dash="dash"),
                visible=False,
            )
        )

    # Make the first trace visible
    fig.data[0].visible = True
    fig.data[1].visible = True

    # Create dropdown menu
    dropdown_buttons = []
    for i, col in enumerate(forecasts.keys()):
        dress_id = col.split("_")[1]
        dress_attributes = final_data.loc[final_data["Dress_ID"] == int(dress_id)].iloc[
            0
        ]
        attributes_text = f"Material: {dress_attributes['Material']}, Sleeve Length: {dr

        dropdown_buttons.append(
            dict(
                label=col,
                method="update",
                args=[
                    {"visible": [False] * len(fig.data)},
                    {
                        "title": f"Sales Forecast for Dress ID {dress_id} ({attributes_t
                    },
                ],
            )
        )
        dropdown_buttons[-1]["args"][0]["visible"][2 * i] = True
        dropdown_buttons[-1]["args"][0]["visible"][2 * i + 1] = True

    fig.update_layout(
        updatemenus=[dict(active=0, buttons=dropdown_buttons)],
        title="Sales Forecast",
```

```
        xaxis_title="Date",
        yaxis_title="Sales",
    )

    fig.show()
```

In [ ]:
```
# Create interactive plot
create_interactive_plot()
```

Interpretation of Results

- **ADF Statistic and p-value**: The Dickey-Fuller test results indicate whether the series is stationary. A lower p-value (< 0.05) suggests that the series is stationary.
- **Best ARIMA Order**: The grid search identifies the best ARIMA order (p, d, q) based on RMSE.
- **Model Summary**: The ARIMA model summary provides details about the fitted model, including coefficients and diagnostics.
- **RMSE**: The Root Mean Squared Error (RMSE) on the testing set indicates the model's forecasting accuracy.

Conclusion and Recommendations

Based on the analysis and model evaluation:

- **Stationarity**: Ensure the time series data is stationary before fitting the ARIMA model.
- **Model Selection**: Use grid search to identify the best ARIMA order for each 'Sales_' column.
- **Model Performance**: Evaluate the model's performance using RMSE and adjust the model parameters as needed.
- **Future Work**: Explore other time series models and techniques to improve forecasting accuracy.

Enhancements:

-Since the above model gives the RMSE of each Sales *column which is essentially the sales for each unique Dress_ID, let's generate forecasts for each Sales* column and combine them in a data frame. From there we can see how to combine the forecasts to predict which Dress attributes would result in the best sales as well for further enhancements on ARIMA.

Fit ARIMA Model using the best order on the training set and generate forecasts

TO BE IMPLEMENTED -- since model took 5 hours or more to run, the new forecast enhancement into a date frame is going to have to wait until model is finished running.